

read 函数从描述符为 fd 的当前文件位置复制最多 n 个字节到内存位置 buf。返回值 -1 表示一个错误，而返回值 0 表示 EOF。否则，返回值表示的是实际传送的字节数量。

write 函数从内存位置 buf 复制至多 n 个字节到描述符 fd 的当前文件位置。图 10-3 展示了一个程序使用 read 和 write 调用一次一个字节地从标准输入复制到标准输出。

```

1  #include "csapp.h"
2
3  int main(void)
4  {
5      char c;
6
7      while(Read(STDIN_FILENO, &c, 1) != 0)
8          Write(STDOUT_FILENO, &c, 1);
9      exit(0);
10 }

```

code/io/cpstdin.c

code/io/cpstdin.c

图 10-3 一次一个字节地从标准输入复制到标准输出

通过调用 lseek 函数，应用程序能够显示地修改当前文件的位置，这部分内容不在我们的讲述范围之内。

旁注 ssize_t 和 size_t 有些什么区别？

你可能已经注意到了，read 函数有一个 size_t 的输入参数和一个 ssize_t 的返回值。那么这两种类型之间有什么区别呢？在 x86-64 系统中，size_t 被定义为 unsigned long，而 ssize_t(有符号的大小)被定义为 long。read 函数返回一个有符号的大小，而不是一个无符号大小，这是因为出错时它必须返回 -1。有趣的是，返回一个 -1 的可能性使得 read 的最大值减小了一半。

在某些情况下，read 和 write 传送的字节比应用程序要求的要少。这些不足值(short count)不表示有错误。出现这样情况的原因有：

- 读时遇到 EOF。假设我们准备读一个文件，该文件从当前文件位置开始只含有 20 多个字节，而我们以 50 个字节的片进行读取。这样一来，下一个 read 返回的不足值为 20，此后的 read 将通过返回不足值 0 来发出 EOF 信号。
- 从终端读文本行。如果打开文件是与终端相关联的(如键盘和显示器)，那么每个 read 函数将一次传送一个文本行，返回的不足值等于文本行的大小。
- 读和写网络套接字(socket)。如果打开的文件对应于网络套接字(11.4 节)，那么内部缓冲约束和较长的网络延迟会引起 read 和 write 返回不足值。对 Linux 管道(pipe)调用 read 和 write 时，也有可能出现不足值，这种进程间通信机制不在我们讨论的范围之内。

实际上，除了 EOF，当你在读磁盘文件时，将不会遇到不足值，而且在写磁盘文件时，也不会遇到不足值。然而，如果你想创建健壮的(可靠的)诸如 Web 服务器这样的网络应用，就必须通过反复调用 read 和 write 处理不足值，直到所有需要的字节都传送完毕。

10.5 用 RIO 包健壮地读写

在这一小节里，我们会讲述一个 I/O 包，称为 RIO(Robust I/O，健壮的 I/O)包，它

会自动为你处理上文中所述的不足值。在像网络程序这样容易出现不足值的应用中，RIO 包提供了方便、健壮和高效的 I/O。RIO 提供了两类不同的函数：

- 无缓冲的输入输出函数。这些函数直接在内存和文件之间传送数据，没有应用级缓冲。它们对将二进制数据读写到网络和从网络读写二进制数据尤其有用。
- 带缓冲的输入函数。这些函数允许你高效地从文件中读取文本行和二进制数据，这些文件的内容缓存在应用级缓冲区内，类似于为 `printf` 这样的标准 I/O 函数提供的缓冲区。与 [110] 中讲述的带缓冲的 I/O 例程不同，带缓冲的 RIO 输入函数是线程安全的 (12.7.1 节)，它在同一个描述符上可以被交错地调用。例如，你可以从一个描述符中读一些文本行，然后读取一些二进制数据，接着再多读取一些文本行。

我们讲述 RIO 例程有两个原因。第一，在接下来的两章中，我们开发的网络应用中使用了它们；第二，通过学习这些例程的代码，你将从总体上对 Unix I/O 有更深入的了解。

10.5.1 RIO 的无缓冲的输入输出函数

通过调用 `rio_readn` 和 `rio_writen` 函数，应用程序可以在内存和文件之间直接传送数据。

```
#include "csapp.h"
```

```
ssize_t rio_readn(int fd, void *usrbuf, size_t n);
ssize_t rio_writen(int fd, void *usrbuf, size_t n);
```

返回：若成功则为传送的字节数，若 EOF 则为 0 (只对 `rio_readn` 而言)，若出错则为 -1。

`rio_readn` 函数从描述符 `fd` 的当前文件位置最多传送 n 个字节到内存位置 `usrbuf`。类似地，`rio_writen` 函数从位置 `usrbuf` 传送 n 个字节到描述符 `fd`。`rio_read` 函数在遇到 EOF 时只能返回一个不足值。`rio_writen` 函数决不会返回不足值。对同一个描述符，可以任意交错地调用 `rio_readn` 和 `rio_writen`。

图 10-4 显示了 `rio_readn` 和 `rio_writen` 的代码。注意，如果 `rio_readn` 和 `rio_writen` 函数被一个从应用信号处理程序的返回中断，那么每个函数都会手动地重启 `read` 或 `write`。为了尽可能有较好的可移植性，我们允许被中断的系统调用，且在必要时重启它们。

10.5.2 RIO 的带缓冲的输入函数

假设我们要编写一个程序来计算文本文件中文本行的数量，该如何来实现呢？一种方法就是用 `read` 函数来一次一个字节地从文件传送到用户内存，检查每个字节来查找换行符。这个方法的缺点是效率不是很高，每读取文件中的一个字节都要求陷入内核。

一种更好的方法是调用一个包装函数 (`rio_readlineb`)，它从一个内部读缓冲区复制一个文本行，当缓冲区变空时，会自动地调用 `read` 重新填满缓冲区。对于既包含文本行也包含二进制数据的文件 (例如 11.5.3 节中描述的 HTTP 响应)，我们也提供了一个 `rio_readn` 带缓冲区的版本，叫做 `rio_readnb`，它从和 `rio_readlineb` 一样的读缓冲区中传送原始字节。

```
#include "csapp.h"
```

```
void rio_readinitb(rio_t *rp, int fd);
```

返回：无。

```
ssize_t rio_readlineb(rio_t *rp, void *usrbuf, size_t maxlen);
ssize_t rio_readnb(rio_t *rp, void *usrbuf, size_t n);
```

返回：若成功则为读的字节数，若 EOF 则为 0，若出错则为 -1。

code/src/csapp.c

```

1  ssize_t rio_readn(int fd, void *usrbuf, size_t n)
2  {
3      size_t nleft = n;
4      ssize_t nread;
5      char *bufp = usrbuf;
6
7      while (nleft > 0) {
8          if ((nread = read(fd, bufp, nleft)) < 0) {
9              if (errno == EINTR) /* Interrupted by sig handler return */
10                 nread = 0;      /* and call read() again */
11              else
12                 return -1;      /* errno set by read() */
13          }
14          else if (nread == 0)
15              break;             /* EOF */
16          nleft -= nread;
17          bufp += nread;
18      }
19      return (n - nleft);        /* Return >= 0 */
20  }

```

*code/src/csapp.c**code/src/csapp.c*

```

1  ssize_t rio_writen(int fd, void *usrbuf, size_t n)
2  {
3      size_t nleft = n;
4      ssize_t nwritten;
5      char *bufp = usrbuf;
6
7      while (nleft > 0) {
8          if ((nwritten = write(fd, bufp, nleft)) <= 0) {
9              if (errno == EINTR) /* Interrupted by sig handler return */
10                 nwritten = 0;    /* and call write() again */
11              else
12                 return -1;      /* errno set by write() */
13          }
14          nleft -= nwritten;
15          bufp += nwritten;
16      }
17      return n;
18  }

```

code/src/csapp.c

图 10-4 rio_readn 和 rio_writen 函数

每打开一个描述符，都会调用一次 `rio_readinitb` 函数。它将描述符 `fd` 和地址 `rp` 处的一个类型为 `rio_t` 的读缓冲区联系起来。

`rio_readlineb` 函数从文件 `rp` 读出下一个文本行(包括结尾的换行符)，将它复制到内存位置 `usrbuf`，并且用 `NULL`(零)字符来结束这个文本行。`rio_readlineb` 函数最多读 `maxlen-1` 个字节，余下的一个字符留给结尾的 `NULL` 字符。超过 `maxlen-1` 字节的文

本行被截断，并用一个 NULL 字符结束。

`rio_readnb` 函数从文件 `rp` 最多读 n 个字节到内存位置 `usrbuf`。对同一描述符，对 `rio_readlineb` 和 `rio_readnb` 的调用可以任意交叉进行。然而，对这些带缓冲的函数的调用却不应和无缓冲的 `rio_readn` 函数交叉使用。

在本书剩下的部分中将给出大量的 RIO 函数的示例。图 10-5 展示了如何使用 RIO 函数来一次一行地从标准输入复制一个文本文件到标准输出。

```

1  #include "csapp.h"
2
3  int main(int argc, char **argv)
4  {
5      int n;
6      rio_t rio;
7      char buf[MAXLINE];
8
9      Rio_readinitb(&rio, STDIN_FILENO);
10     while((n = Rio_readlineb(&rio, buf, MAXLINE)) != 0)
11         Rio_writen(STDOUT_FILENO, buf, n);
12 }

```

code/io/cpfile.c

图 10-5 从标准输入复制一个文本文件到标准输出

图 10-6 展示了一个读缓冲区的格式，以及初始化它的 `rio_readinitb` 函数的代码。`rio_readinitb` 函数创建了一个空的读缓冲区，并且将一个打开的文件描述符和这个缓冲区联系起来。

```

1  #define RIO_BUFSIZE 8192
2  typedef struct {
3      int rio_fd;           /* Descriptor for this internal buf */
4      int rio_cnt;          /* Unread bytes in internal buf */
5      char *rio_bufptr;     /* Next unread byte in internal buf */
6      char rio_buf[RIO_BUFSIZE]; /* Internal buffer */
7  } rio_t;

```

code/include/csapp.h

```

1  void rio_readinitb(rio_t *rp, int fd)
2  {
3      rp->rio_fd = fd;
4      rp->rio_cnt = 0;
5      rp->rio_bufptr = rp->rio_buf;
6  }

```

code/src/csapp.c

code/src/csapp.c

图 10-6 一个类型为 `rio_t` 的读缓冲区和初始化它的 `rio_readinitb` 函数

RIO 读程序的核心是图 10-7 所示的 `rio_read` 函数。`rio_read` 函数是 Linux `read` 函数的带缓冲的版本。当调用 `rio_read` 要求读 n 个字节时，读缓冲区内有 `rp->rio_cnt`

个未读字节。如果缓冲区为空，那么会通过调用 `read` 再填满它。这个 `read` 调用收到一个不足值并不是错误，只不过读缓冲区是填充了一部分。一旦缓冲区非空，`rio_read` 就从读缓冲区复制 `n` 和 `rp->rio_cnt` 中较小值个字节到用户缓冲区，并返回复制的字节数。

code/src/csapp.c

```

1  static ssize_t rio_read(rio_t *rp, char *usrbuf, size_t n)
2  {
3      int cnt;
4
5      while (rp->rio_cnt <= 0) { /* Refill if buf is empty */
6          rp->rio_cnt = read(rp->rio_fd, rp->rio_buf,
7                          sizeof(rp->rio_buf));
8          if (rp->rio_cnt < 0) {
9              if (errno != EINTR) /* Interrupted by sig handler return */
10                 return -1;
11          }
12          else if (rp->rio_cnt == 0) /* EOF */
13              return 0;
14          else
15              rp->rio_bufptr = rp->rio_buf; /* Reset buffer ptr */
16      }
17
18      /* Copy min(n, rp->rio_cnt) bytes from internal buf to user buf */
19      cnt = n;
20      if (rp->rio_cnt < n)
21          cnt = rp->rio_cnt;
22      memcpy(usrbuf, rp->rio_bufptr, cnt);
23      rp->rio_bufptr += cnt;
24      rp->rio_cnt -= cnt;
25      return cnt;
26  }
```

code/src/csapp.c

图 10-7 内部的 `rio_read` 函数

对于一个应用程序，`rio_read` 函数和 Linux `read` 函数有同样的语义。在出错时，它返回值 `-1`，并且适当地设置 `errno`。在 EOF 时，它返回值 `0`。如果要求的字节数超过了读缓冲区内未读的字节的数量，它会返回一个不足值。两个函数的相似性使得很容易通过用 `rio_read` 代替 `read` 来创建不同类型的带缓冲的读函数。例如，用 `rio_read` 代替 `read`，图 10-8 中的 `rio_readnb` 函数和 `rio_readn` 有相同的结构。相似地，图 10-8 中的 `rio_readlineb` 程序最多调用 `maxlen-1` 次 `rio_read`。每次调用都从读缓冲区返回一个字节，然后检查这个字节是否是结尾的换行符。

旁注 RIO 包的起源

RIO 函数的灵感来自于 W. Richard Stevens 在他的经典网络编程作品[110]中描述的 `readline`、`readn` 和 `writen` 函数。`rio_readn` 和 `rio_writen` 函数与 Stevens 的 `readn` 和 `writen` 函数是一样的。然而，Stevens 的 `readline` 函数有一些局限性在 RIO 中得到了纠正。第一，因为 `readline` 是带缓冲的，而 `readn` 不带，所以这两个函数不能在同一描述符上一起使用。第二，因为它使用一个 `static` 缓冲区，Stevens 的 `readline`

函数不是线程安全的，这就要求 Stevens 引入一个不同的线程安全的版本，称为 `readline_r`。我们已经在 `rio_readlineb` 和 `rio_readnb` 函数中修改了这两个缺陷，使得这两个函数是相互兼容和线程安全的。

code/src/csapp.c

```

1  ssize_t rio_readlineb(rio_t *rp, void *usrbuf, size_t maxlen)
2  {
3      int n, rc;
4      char c, *bufp = usrbuf;
5
6      for (n = 1; n < maxlen; n++) {
7          if ((rc = rio_read(rp, &c, 1)) == 1) {
8              *bufp++ = c;
9              if (c == '\n') {
10                 n++;
11                 break;
12             }
13         } else if (rc == 0) {
14             if (n == 1)
15                 return 0; /* EOF, no data read */
16             else
17                 break; /* EOF, some data was read */
18         } else
19             return -1; /* Error */
20     }
21     *bufp = 0;
22     return n-1;
23 }
```

code/src/csapp.c

code/src/csapp.c

```

1  ssize_t rio_readnb(rio_t *rp, void *usrbuf, size_t n)
2  {
3      size_t nleft = n;
4      ssize_t nread;
5      char *bufp = usrbuf;
6
7      while (nleft > 0) {
8          if ((nread = rio_read(rp, bufp, nleft)) < 0)
9              return -1; /* errno set by read() */
10         else if (nread == 0)
11             break; /* EOF */
12         nleft -= nread;
13         bufp += nread;
14     }
15     return (n - nleft); /* Return >= 0 */
16 }
```

code/src/csapp.c

图 10-8 `rio_readlineb` 和 `rio_readnb` 函数

10.6 读取文件元数据

应用程序能够通过调用 `stat` 和 `fstat` 函数，检索到关于文件的信息（有时也称为文件的元数据(metadata)）。

```
#include <unistd.h>
#include <sys/stat.h>

int stat(const char *filename, struct stat *buf);
int fstat(int fd, struct stat *buf);
```

返回：若成功则为 0，若出错则为 -1。

`stat` 函数以一个文件名作为输入，并填写如图 10-9 所示的一个 `stat` 数据结构中的各个成员。`fstat` 函数是相似的，只不过是以文件描述符而不是文件名作为输入。当我们在 11.5 节中讨论 Web 服务器时，会需要 `stat` 数据结构中的 `st_mode` 和 `st_size` 成员，其他成员则不在我们的讨论之列。

```
----- statbuf.h (included by sys/stat.h)

/* Metadata returned by the stat and fstat functions */
struct stat {
    dev_t      st_dev;      /* Device */
    ino_t      st_ino;      /* inode */
    mode_t     st_mode;     /* Protection and file type */
    nlink_t    st_nlink;    /* Number of hard links */
    uid_t      st_uid;     /* User ID of owner */
    gid_t      st_gid;     /* Group ID of owner */
    dev_t      st_rdev;     /* Device type (if inode device) */
    off_t      st_size;     /* Total size, in bytes */
    unsigned long st_blksize; /* Block size for filesystem I/O */
    unsigned long st_blocks; /* Number of blocks allocated */
    time_t     st_atime;    /* Time of last access */
    time_t     st_mtime;    /* Time of last modification */
    time_t     st_ctime;    /* Time of last change */
};

----- statbuf.h (included by sys/stat.h)
```

图 10-9 stat 数据结构

`st_size` 成员包含了文件的字节数大小。`st_mode` 成员则编码了文件访问许可位(图 10-2)和文件类型(10.2 节)。Linux 在 `sys/stat.h` 中定义了宏谓词来确定 `st_mode` 成员的文件类型：

`S_ISREG(m)`。这是一个普通文件吗？

`S_ISDIR(m)`。这是一个目录文件吗？

`S_ISSOCK(m)`。这是一个网络套接字吗？

图 10-10 展示了我们会如何使用这些宏和 `stat` 函数来读取和解释一个文件的 `st_mode` 位。